

Data Structures & Algorithms - 50 Interview Questions with Answers

Arrays & Strings (Questions 1-10)

Q1. Find the maximum subarray sum (Kadane's Algorithm).

Answer:

```
def max_subarray_sum(arr):
    max_current = max_global = arr[0]
    for i in range(1, len(arr)):
        max_current = max(arr[i], max_current + arr[i])
        max_global = max(max_global, max_current)
    return max_global
```

Time: O(n), **Space:** O(1)

Q2. Reverse a string in-place.

Answer:

```
def reverse_string(s):
    return s[::-1] # Python way

# Or manual:
def reverse_manual(s):
    chars = list(s)
    left, right = 0, len(chars) - 1
    while left < right:
        chars[left], chars[right] = chars[right], chars[left]
        left += 1
        right -= 1
    return ''.join(chars)
```

Time: O(n), **Space:** O(1)

Q3. Find duplicate in array of n+1 integers (1 to n).

Answer: Floyd's Cycle Detection

```
def find_duplicate(nums):
    slow = fast = nums[0]
    while True:
        slow = nums[slow]
```

```
        fast = nums[nums[fast]]
        if slow == fast:
            break

    slow = nums[0]
    while slow != fast:
        slow = nums[slow]
        fast = nums[fast]
    return slow
```

Time: $O(n)$, **Space:** $O(1)$

Q4. Rotate array right by k steps.

Answer:

```
def rotate(nums, k):
    k = k % len(nums)
    # Reverse entire array
    nums.reverse()
    # Reverse first k
    nums[:k] = reversed(nums[:k])
    # Reverse remaining
    nums[k:] = reversed(nums[k:])
```

Time: $O(n)$, **Space:** $O(1)$

Q5. Find all pairs with given sum in array.

Answer:

```
def find_pairs(arr, target):
    seen = set()
    pairs = []
    for num in arr:
        complement = target - num
        if complement in seen:
            pairs.append((complement, num))
        seen.add(num)
    return pairs
```

Time: $O(n)$, **Space:** $O(n)$

Q6. Longest substring without repeating characters.

Answer: Sliding Window

```
def length_of_longest_substring(s):
    char_set = set()
    left = max_length = 0

    for right in range(len(s)):
        while s[right] in char_set:
            char_set.remove(s[left])
            left += 1
        char_set.add(s[right])
        max_length = max(max_length, right - left + 1)

    return max_length
```

Time: $O(n)$, **Space:** $O(\min(n, m))$ where m is charset size

Q7. Merge two sorted arrays.

Answer:

```
def merge_sorted(arr1, arr2):
    i = j = 0
    result = []

    while i < len(arr1) and j < len(arr2):
        if arr1[i] < arr2[j]:
            result.append(arr1[i])
            i += 1
        else:
            result.append(arr2[j])
            j += 1

    result.extend(arr1[i:])
    result.extend(arr2[j:])
    return result
```

Time: $O(n+m)$, **Space:** $O(n+m)$

Q8. Find missing number in array 1 to n .

Answer:

```
def find_missing(nums):
    n = len(nums) + 1
    expected_sum = n * (n + 1) // 2
    actual_sum = sum(nums)
    return expected_sum - actual_sum
```

Time: $O(n)$, **Space:** $O(1)$

Q9. Move all zeros to end of array.

Answer:

```
def move_zeros(nums):
    left = 0
    for right in range(len(nums)):
        if nums[right] != 0:
            nums[left], nums[right] = nums[right], nums[left]
            left += 1
```

Time: $O(n)$, **Space:** $O(1)$

Q10. Check if string is palindrome.

Answer:

```
def is_palindrome(s):
    # Clean string
    s = ''.join(c.lower() for c in s if c.isalnum())
    return s == s[::-1]
```

Time: $O(n)$, **Space:** $O(n)$

Linked Lists (Questions 11-15)

Q11. Reverse a linked list.

Answer:

```
def reverse_list(head):
    prev = None
    current = head

    while current:
        next_node = current.next
        current.next = prev
        prev = current
        current = next_node

    return prev
```

Time: $O(n)$, **Space:** $O(1)$

Q12. Detect cycle in linked list.

Answer: Floyd's Algorithm

```
def has_cycle(head):  
    if not head:  
        return False  
  
    slow = fast = head  
    while fast and fast.next:  
        slow = slow.next  
        fast = fast.next.next  
        if slow == fast:  
            return True  
  
    return False
```

Time: $O(n)$, **Space:** $O(1)$

Q13. Find middle of linked list.

Answer:

```
def find_middle(head):  
    slow = fast = head  
    while fast and fast.next:  
        slow = slow.next  
        fast = fast.next.next  
    return slow
```

Time: $O(n)$, **Space:** $O(1)$

Q14. Merge two sorted linked lists.

Answer:

```
def merge_two_lists(l1, l2):  
    dummy = ListNode(0)  
    current = dummy  
  
    while l1 and l2:  
        if l1.val < l2.val:  
            current.next = l1  
            l1 = l1.next  
        else:  
            current.next = l2  
            l2 = l2.next  
        current = current.next
```

```
current.next = l1 or l2
return dummy.next
```

Time: $O(n+m)$, **Space:** $O(1)$

Q15. Remove nth node from end.

Answer:

```
def remove_nth_from_end(head, n):
    dummy = ListNode(0)
    dummy.next = head
    first = second = dummy

    # Move first n+1 steps ahead
    for _ in range(n + 1):
        first = first.next

    # Move both until first reaches end
    while first:
        first = first.next
        second = second.next

    # Remove node
    second.next = second.next.next
    return dummy.next
```

Time: $O(n)$, **Space:** $O(1)$

Stacks & Queues (Questions 16-20)

Q16. Implement stack using queues.

Answer:

```
from collections import deque

class Stack:
    def __init__(self):
        self.q = deque()

    def push(self, x):
        self.q.append(x)
        # Rotate to make last element first
        for _ in range(len(self.q) - 1):
            self.q.append(self.q.popleft())

    def pop(self):
        return self.q.popleft()
```

```
def top(self):  
    return self.q[0]
```

Time: Push $O(n)$, Pop/Top $O(1)$

Q17. Valid parentheses check.

Answer:

```
def is_valid(s):  
    stack = []  
    mapping = {'(': ')', '(': ')', '{': '{', '[': '['}  
  
    for char in s:  
        if char in mapping:  
            top = stack.pop() if stack else '#'  
            if mapping[char] != top:  
                return False  
        else:  
            stack.append(char)  
  
    return not stack
```

Time: $O(n)$, **Space:** $O(n)$

Q18. Implement queue using stacks.

Answer:

```
class Queue:  
    def __init__(self):  
        self.s1 = [] # for enqueue  
        self.s2 = [] # for dequeue  
  
    def enqueue(self, x):  
        self.s1.append(x)  
  
    def dequeue(self):  
        if not self.s2:  
            while self.s1:  
                self.s2.append(self.s1.pop())  
        return self.s2.pop() if self.s2 else None
```

Time: Amortized $O(1)$

Q19. Next greater element.

Answer:

```
def next_greater_element(nums):
    result = [-1] * len(nums)
    stack = []

    for i in range(len(nums) - 1, -1, -1):
        while stack and stack[-1] <= nums[i]:
            stack.pop()

        result[i] = stack[-1] if stack else -1
        stack.append(nums[i])

    return result
```

Time: O(n), **Space:** O(n)

Q20. Min stack (with getMin in O(1)).

Answer:

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.min_stack = []

    def push(self, x):
        self.stack.append(x)
        if not self.min_stack or x <= self.min_stack[-1]:
            self.min_stack.append(x)

    def pop(self):
        if self.stack.pop() == self.min_stack[-1]:
            self.min_stack.pop()

    def get_min(self):
        return self.min_stack[-1]
```

Time: O(1) all operations

Trees & Graphs (Questions 21-30)

Q21. Binary tree inorder traversal.

Answer:

```
def inorder(root):
    result = []
```

```
def traverse(node):
    if not node:
        return
    traverse(node.left)
    result.append(node.val)
    traverse(node.right)

traverse(root)
return result
```

Time: $O(n)$, **Space:** $O(h)$ where h is height

Q22. Level order traversal (BFS).

Answer:

```
from collections import deque

def level_order(root):
    if not root:
        return []

    result = []
    queue = deque([root])

    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level)

    return result
```

Time: $O(n)$, **Space:** $O(n)$

Q23. Check if binary tree is balanced.

Answer:

```
def is_balanced(root):
    def height(node):
        if not node:
            return 0
```

```

    left = height(node.left)
    right = height(node.right)

    if left == -1 or right == -1 or abs(left - right) > 1:
        return -1

    return max(left, right) + 1

return height(root) != -1

```

Time: $O(n)$, **Space:** $O(h)$

Q24. Lowest common ancestor in BST.

Answer:

```

def lowest_common_ancestor(root, p, q):
    if p.val < root.val and q.val < root.val:
        return lowest_common_ancestor(root.left, p, q)
    elif p.val > root.val and q.val > root.val:
        return lowest_common_ancestor(root.right, p, q)
    else:
        return root

```

Time: $O(h)$, **Space:** $O(h)$

Q25. Diameter of binary tree.

Answer:

```

def diameter_of_binary_tree(root):
    diameter = 0

    def height(node):
        nonlocal diameter
        if not node:
            return 0

        left = height(node.left)
        right = height(node.right)
        diameter = max(diameter, left + right)

        return max(left, right) + 1

    height(root)
    return diameter

```

Time: $O(n)$, **Space:** $O(h)$

Q26. Graph DFS traversal.

Answer:

```
def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()

    visited.add(start)
    result = [start]

    for neighbor in graph[start]:
        if neighbor not in visited:
            result.extend(dfs(graph, neighbor, visited))

    return result
```

Time: $O(V+E)$, **Space:** $O(V)$

Q27. Graph BFS traversal.

Answer:

```
from collections import deque

def bfs(graph, start):
    visited = set([start])
    queue = deque([start])
    result = []

    while queue:
        vertex = queue.popleft()
        result.append(vertex)

        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

    return result
```

Time: $O(V+E)$, **Space:** $O(V)$

Q28. Detect cycle in directed graph.

Answer:

```
def has_cycle(graph):
    WHITE, GRAY, BLACK = 0, 1, 2
    color = {node: WHITE for node in graph}

    def dfs(node):
        if color[node] == GRAY:
            return True # Back edge found
        if color[node] == BLACK:
            return False

        color[node] = GRAY
        for neighbor in graph[node]:
            if dfs(neighbor):
                return True

        color[node] = BLACK
        return False

    for node in graph:
        if color[node] == WHITE:
            if dfs(node):
                return True
    return False
```

Time: $O(V+E)$, **Space:** $O(V)$

Q29. Number of islands (2D grid).

Answer:

```
def num_islands(grid):
    if not grid:
        return 0

    count = 0
    rows, cols = len(grid), len(grid[0])

    def dfs(r, c):
        if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] != '1':
            return
        grid[r][c] = '0' # Mark visited
        dfs(r+1, c)
        dfs(r-1, c)
        dfs(r, c+1)
        dfs(r, c-1)

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                count += 1
                dfs(r, c)
```

```
return count
```

Time: $O(m \times n)$, **Space:** $O(m \times n)$

Q30. Shortest path in unweighted graph (BFS).

Answer:

```
from collections import deque

def shortest_path(graph, start, end):
    queue = deque([(start, [start])])
    visited = set([start])

    while queue:
        node, path = queue.popleft()

        if node == end:
            return path

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [neighbor]))

    return None
```

Time: $O(V+E)$, **Space:** $O(V)$

Sorting & Searching (Questions 31-40)

Q31. Quick Sort implementation.

Answer:

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return quick_sort(left) + middle + quick_sort(right)
```

Time: Average $O(n \log n)$, Worst $O(n^2)$

Q32. Merge Sort implementation.

Answer:

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

Time: $O(n \log n)$, **Space:** $O(n)$

Q33. Binary search implementation.

Answer:

```
def binary_search(arr, target):
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = left + (right - left) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1
```

Time: $O(\log n)$, **Space:** $O(1)$

Q34. Find first and last position of element in sorted array.

Answer:

```
def search_range(nums, target):
    def find_bound(is_first):
        left, right = 0, len(nums) - 1
        result = -1

        while left <= right:
            mid = left + (right - left) // 2

            if nums[mid] == target:
                result = mid
                if is_first:
                    right = mid - 1
                else:
                    left = mid + 1
            elif nums[mid] < target:
                left = mid + 1
            else:
                right = mid - 1

        return result

    return [find_bound(True), find_bound(False)]
```

Time: $O(\log n)$, **Space:** $O(1)$

Q35. Search in rotated sorted array.

Answer:

```
def search_rotated(nums, target):
    left, right = 0, len(nums) - 1

    while left <= right:
        mid = left + (right - left) // 2

        if nums[mid] == target:
            return mid

        # Left half is sorted
        if nums[left] <= nums[mid]:
            if nums[left] <= target < nums[mid]:
                right = mid - 1
            else:
                left = mid + 1
        else:
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1
```

```
        left = mid + 1
    # Right half is sorted
    else:
        if nums[mid] < target <= nums[right]:
            left = mid + 1
        else:
            right = mid - 1

    return -1
```

Time: $O(\log n)$, **Space:** $O(1)$

Q36. Find peak element.

Answer:

```
def find_peak_element(nums):
    left, right = 0, len(nums) - 1

    while left < right:
        mid = left + (right - left) // 2

        if nums[mid] > nums[mid + 1]:
            right = mid
        else:
            left = mid + 1

    return left
```

Time: $O(\log n)$, **Space:** $O(1)$

Q37. Kth largest element.

Answer:

```
import heapq

def find_kth_largest(nums, k):
    return heapq.nlargest(k, nums)[-1]

# Or using min heap:
def find_kth_largest_heap(nums, k):
    heap = nums[:k]
    heapq.heapify(heap)

    for num in nums[k:]:
        if num > heap[0]:
            heapq.heappushpop(heap, num)
```

```
return heap[0]
```

Time: $O(n \log k)$, **Space:** $O(k)$

Q38. Median of two sorted arrays.

Answer:

```
def find_median_sorted_arrays(nums1, nums2):
    if len(nums1) > len(nums2):
        nums1, nums2 = nums2, nums1

    m, n = len(nums1), len(nums2)
    left, right = 0, m

    while left <= right:
        partition1 = (left + right) // 2
        partition2 = (m + n + 1) // 2 - partition1

        max_left1 = float('-inf') if partition1 == 0 else nums1[partition1 - 1]
        min_right1 = float('inf') if partition1 == m else nums1[partition1]

        max_left2 = float('-inf') if partition2 == 0 else nums2[partition2 - 1]
        min_right2 = float('inf') if partition2 == n else nums2[partition2]

        if max_left1 <= min_right2 and max_left2 <= min_right1:
            if (m + n) % 2 == 0:
                return (max(max_left1, max_left2) + min(min_right1, min_right2)) / 2
            else:
                return max(max_left1, max_left2)
        elif max_left1 > min_right2:
            right = partition1 - 1
        else:
            left = partition1 + 1
```

Time: $O(\log(\min(m,n)))$, **Space:** $O(1)$

Q39. Count inversions in array.

Answer: Using merge sort

```
def count_inversions(arr):
    def merge_count(arr, temp, left, mid, right):
        i, j, k = left, mid + 1, left
        inv_count = 0

        while i <= mid and j <= right:
```

```

        if arr[i] <= arr[j]:
            temp[k] = arr[i]
            i += 1
        else:
            temp[k] = arr[j]
            inv_count += (mid - i + 1)
            j += 1
        k += 1

    while i <= mid:
        temp[k] = arr[i]
        i += 1
        k += 1

    while j <= right:
        temp[k] = arr[j]
        j += 1
        k += 1

    for i in range(left, right + 1):
        arr[i] = temp[i]

    return inv_count

def merge_sort_count(arr, temp, left, right):
    inv_count = 0
    if left < right:
        mid = (left + right) // 2
        inv_count += merge_sort_count(arr, temp, left, mid)
        inv_count += merge_sort_count(arr, temp, mid + 1, right)
        inv_count += merge_count(arr, temp, left, mid, right)
    return inv_count

n = len(arr)
temp = [0] * n
return merge_sort_count(arr, temp, 0, n - 1)

```

Time: $O(n \log n)$, **Space:** $O(n)$

Q40. Sort colors (Dutch flag problem).

Answer:

```

def sort_colors(nums):
    low, mid, high = 0, 0, len(nums) - 1

    while mid <= high:
        if nums[mid] == 0:
            nums[low], nums[mid] = nums[mid], nums[low]
            low += 1
            mid += 1

```

```
elif nums[mid] == 1:
    mid += 1
else:
    nums[mid], nums[high] = nums[high], nums[mid]
    high -= 1
```

Time: $O(n)$, **Space:** $O(1)$

Dynamic Programming (Questions 41-50)

Q41. Fibonacci number.

Answer:

```
def fibonacci(n):
    if n <= 1:
        return n

    dp = [0] * (n + 1)
    dp[1] = 1

    for i in range(2, n + 1):
        dp[i] = dp[i-1] + dp[i-2]

    return dp[n]

# Space optimized:
def fibonacci_optimized(n):
    if n <= 1:
        return n

    prev, curr = 0, 1
    for _ in range(2, n + 1):
        prev, curr = curr, prev + curr

    return curr
```

Time: $O(n)$, **Space:** $O(1)$

Q42. Climbing stairs.

Answer:

```
def climb_stairs(n):
    if n <= 2:
        return n

    prev, curr = 1, 2
    for _ in range(3, n + 1):
```

```
    prev, curr = curr, prev + curr

    return curr
```

Time: $O(n)$, **Space:** $O(1)$

Q43. Longest increasing subsequence.

Answer:

```
def length_of_LIS(nums):
    if not nums:
        return 0

    dp = [1] * len(nums)

    for i in range(1, len(nums)):
        for j in range(i):
            if nums[i] > nums[j]:
                dp[i] = max(dp[i], dp[j] + 1)

    return max(dp)
```

Time: $O(n^2)$, **Space:** $O(n)$

Q44. Coin change (minimum coins).

Answer:

```
def coin_change(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0

    for coin in coins:
        for i in range(coin, amount + 1):
            dp[i] = min(dp[i], dp[i - coin] + 1)

    return dp[amount] if dp[amount] != float('inf') else -1
```

Time: $O(\text{amount} \times n)$, **Space:** $O(\text{amount})$

Q45. 0/1 Knapsack.

Answer:

```
def knapsack(weights, values, capacity):
    n = len(weights)
```

```

dp = [[0] * (capacity + 1) for _ in range(n + 1)]

for i in range(1, n + 1):
    for w in range(1, capacity + 1):
        if weights[i-1] <= w:
            dp[i][w] = max(
                values[i-1] + dp[i-1][w - weights[i-1]],
                dp[i-1][w]
            )
        else:
            dp[i][w] = dp[i-1][w]

return dp[n][capacity]

```

Time: $O(n \times \text{capacity})$, **Space:** $O(n \times \text{capacity})$

Q46. Edit distance.

Answer:

```

def min_distance(word1, word2):
    m, n = len(word1), len(word2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if word1[i-1] == word2[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(
                    dp[i-1][j],      # delete
                    dp[i][j-1],      # insert
                    dp[i-1][j-1]     # replace
                )

    return dp[m][n]

```

Time: $O(m \times n)$, **Space:** $O(m \times n)$

Q47. Longest common subsequence.

Answer:

```
def longest_common_subsequence(text1, text2):
    m, n = len(text1), len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i-1] == text2[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    return dp[m][n]
```

Time: $O(m \times n)$, **Space:** $O(m \times n)$

Q48. Maximum product subarray.

Answer:

```
def max_product(nums):
    max_prod = min_prod = result = nums[0]

    for num in nums[1:]:
        temp = max(num, max_prod * num, min_prod * num)
        min_prod = min(num, max_prod * num, min_prod * num)
        max_prod = temp
        result = max(result, max_prod)

    return result
```

Time: $O(n)$, **Space:** $O(1)$

Q49. Partition equal subset sum.

Answer:

```
def can_partition(nums):
    total = sum(nums)
    if total % 2:
        return False

    target = total // 2
    dp = [False] * (target + 1)
    dp[0] = True

    for num in nums:
        for i in range(target, num - 1, -1):
            dp[i] = dp[i] or dp[i - num]
```

```
return dp[target]
```

Time: $O(n \times \text{sum})$, **Space:** $O(\text{sum})$

Q50. House robber.

Answer:

```
def rob(nums):
    if not nums:
        return 0
    if len(nums) == 1:
        return nums[0]

    prev, curr = 0, 0
    for num in nums:
        prev, curr = curr, max(curr, prev + num)

    return curr
```

Time: $O(n)$, **Space:** $O(1)$

Time Complexity Cheat Sheet

Algorithm	Best	Average	Worst	Space
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$
DFS/BFS	$O(V+E)$	$O(V+E)$	$O(V+E)$	$O(V)$

Good luck with your DSA interview! 🚀